ASSIGNMENT-11.3

Name: Daniel Anthonello

HT. No: 2303A5138

Batch: 07

Task 1: Smart Contact Manager (Arrays & Linked Lists)

Scenario: SR University's student club needs a Contact Manager Application to store members' names and phone numbers. GitHub Copilot was used to generate search and delete methods for both array-based and linked-list-based implementations.

Prompt: # Implement a contact manager using arrays and linked list with add, search, delete

Code:

task1_contacts.py

```python
# Task 1: Smart Contact Manager

# --- Array-based ---
class ArrayContacts:
    def __init__(self):
        self.contacts = []
    def add(self, name, phone):
        self.contacts.append({'name': name, 'phone': phone})
    def search(self, name):
        return next((c for c in self.contacts if c['name']==name), None)
    def delete(self, name):
        self.contacts = [c for c in self.contacts if c['name']!=name]

# --- Linked List-based ---
class Node:
    def __init__(self, name, phone):
        self.name = name; self.phone = phone; self.next = None

class LinkedContacts:
    def __init__(self): self.head = None
    def add(self, name, phone):
        node = Node(name, phone)
        node.next = self.head; self.head = node
    def search(self, name):
        cur = self.head
        while cur:
            if cur.name == name: return cur
            cur = cur.next
        return None

ac = ArrayContacts()
ac.add('Muneer','9876543210'); ac.add('Ali','9000000001')
print(ac.search('Muneer'))
ac.delete('Ali')
print('After delete:', ac.contacts)
```

```
{'name': 'Muneer', 'phone': '9876543210'}
After delete: [{'name': 'Muneer', 'phone': '9876543210'}]
```

Result:
{'name': 'Muneer', 'phone': '9876543210'}
After delete: [{'name': 'Muneer', 'phone': '9876543210'}]

Observation: Copilot generated both implementations efficiently. Array-based insertion is O(1) (append), but deletion is O(n) due to list traversal. Linked list insertion is O(1) at head, and deletion also O(n) in the worst case but avoids memory reallocation. For a small contact list, arrays are simpler; for frequently changing large datasets, linked lists are more

memory-efficient.

Task 2: Library Book Search System (Queues & Priority Queues)

Scenario: The SRU Library manages book borrow requests. Faculty requests must be prioritized over student requests. Copilot assisted in generating enqueue() and dequeue() methods for both a regular Queue and a Priority Queue.

Prompt: # Implement a Queue and Priority Queue for library book requests, faculty gets priority 1

Code:

task2_library.py

```python
# Task 2: Library Book Request System
import heapq
from collections import deque

# --- Regular Queue (FIFO) ---
class BookQueue:
    def __init__(self): self.q = deque()
    def enqueue(self, req): self.q.append(req)
    def dequeue(self): return self.q.popleft() if self.q else None

# --- Priority Queue (faculty=1, student=2) ---
class PriorityBookQueue:
    def __init__(self): self.pq = []
    def enqueue(self, priority, name, book):
        heapq.heappush(self.pq, (priority, name, book))
    def dequeue(self):
        return heapq.heappop(self.pq) if self.pq else None

pbq = PriorityBookQueue()
pbq.enqueue(2, 'Student-Ravi',  'Python Basics')
pbq.enqueue(1, 'Faculty-Kumar', 'AI Handbook')
pbq.enqueue(2, 'Student-Priya', 'Data Structures')
print(pbq.dequeue())
print(pbq.dequeue())
print(pbq.dequeue())
```

```
(1, 'Faculty-Kumar', 'AI Handbook')
(2, 'Student-Priya', 'Data Structures')
(2, 'Student-Ravi', 'Python Basics')
```

Result:

(1, 'Faculty-Kumar', 'AI Handbook')

(2, 'Student-Priya', 'Data Structures')

(2, 'Student-Ravi', 'Python Basics')

Observation: Copilot correctly used Python's *heapq* module (min-heap) to implement priority queuing. Faculty requests with priority 1 are always served before student requests with priority 2, regardless of insertion order. The regular Queue (deque) correctly processes requests in FIFO order. This demonstrates the practical difference between FIFO and priority-based scheduling.

Task 3: Emergency Help Desk (Stack Implementation)

Scenario: SR University's IT Help Desk resolves tickets using a Last-In, First-Out (LIFO) approach for escalation. Copilot was used to generate push(), pop(), peek(), and helper methods for checking stack status.

Prompt: # Implement a Stack for IT help desk tickets with push, pop, peek, is_empty, is_full

Code:

task3_helpdesk.py

```python
# Task 3: IT Help Desk Ticket Stack

class HelpDeskStack:
    def __init__(self, capacity=10):
        self.stack = []
        self.capacity = capacity
    def push(self, ticket):
        if not self.is_full():
            self.stack.append(ticket)
            print(f'Raised: {ticket}')
    def pop(self):
        if not self.is_empty():
            t = self.stack.pop()
            print(f'Resolved: {t}'); return t
    def peek(self):
        return self.stack[-1] if self.stack else None
    def is_empty(self): return len(self.stack) == 0
    def is_full(self):  return len(self.stack) >= self.capacity

hd = HelpDeskStack()
for t in ['T1-WiFi','T2-Printer','T3-Login','T4-Crash','T5-VPN']:
    hd.push(t)
print('Top ticket:', hd.peek())
hd.pop(); hd.pop()
print('Remaining:', hd.stack)
```

```
Raised: T1-WiFi  Raised: T2-Printer  Raised: T3-Login
Raised: T4-Crash  Raised: T5-VPN
Top ticket: T5-VPN
Resolved: T5-VPN  Resolved: T4-Crash
Remaining: ['T1-WiFi', 'T2-Printer', 'T3-Login']
```

Result:

Raised: T1-WiFi ... T5-VPN | Top ticket: T5-VPN

Resolved: T5-VPN, T4-Crash | Remaining: ['T1-WiFi', 'T2-Printer', 'T3-Login']

Observation: The stack correctly demonstrates LIFO behavior — the last ticket raised (T5-VPN) was the first to be resolved. Copilot added *is_empty()* and *is_full()* guard methods to prevent underflow and overflow errors. The *peek()* method allows inspecting the next ticket without resolving it. All five simulated tickets were handled correctly.

Task 4: Hash Table with Collision Handling (Chaining)

Objective: Implement a Hash Table supporting Insert, Search, and Delete, with collision handling via chaining. Copilot expanded the provided starter class into a fully working implementation with well-commented methods.

Prompt: # Expand this HashTable class with insert, search, delete and chaining for collisions

Code:

task4_hashtable.py

```python
# Task 4: Hash Table with Chaining

class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        idx = self._hash(key)
        for pair in self.table[idx]:
            if pair[0] == key: pair[1] = value; return
        self.table[idx].append([key, value])

    def search(self, key):
        idx = self._hash(key)
        for pair in self.table[idx]:
            if pair[0] == key: return pair[1]
        return None

    def delete(self, key):
        idx = self._hash(key)
        self.table[idx] = [p for p in self.table[idx] if p[0]!=key]

ht = HashTable()
ht.insert('roll','2303A51475'); ht.insert('name','Muneer')
print(ht.search('name'))
ht.delete('roll')
print(ht.search('roll'))
```

```
Muneer
None
```

Result:

Muneer

None

Observation: Copilot generated a clean chaining-based hash table. Each bucket is a list of key-value pairs, handling collisions transparently. The *_hash()* function uses Python's built-in *hash()* for uniform distribution. Insert updates existing keys in-place, avoiding duplicates. Search and Delete both operate in O(1) average time. After deletion, searching for the removed key correctly returns None.

Task 5: Real-Time Application – Campus Resource Management System

Scenario: Design a Campus Resource Management System covering Student Attendance Tracking, Event Registration, Library Book Borrowing, Bus Scheduling, and Cafeteria Order Queue. The most suitable data structure was chosen for each feature and one was implemented in full.

Feature	Data Structure	Justification
Attendance Tracking	Array (2D)	Fixed rows (students) × columns (dates); O(1) acce ss.
Event Registration	Queue (FIFO)	Registrations processed in order of arrival.
Library Borrowing	Priority Queue	Faculty requests served before students.
Bus Scheduling	Circular Queue	Repeating routes modelled as circular buffer.
Cafeteria Orders	Queue (FIFO)	Orders prepared in the sequence they arrive.

Implemented Feature: Cafeteria Order Queue
Prompt: # Implement cafeteria order management using a queue with place_order, serve_order, status
Code:

task5_cafeteria.py

```python
# Task 5: Cafeteria Order Queue
from collections import deque

class CafeteriaQueue:
    def __init__(self): self.orders = deque()

    def place_order(self, student, item):
        self.orders.append({'student': student, 'item': item})
        print(f'Order placed: {student} -> {item}')

    def serve_order(self):
        if self.orders:
            o = self.orders.popleft()
            print(f'Serving: {o["student"]} -> {o["item"]}')
        else:
            print('No pending orders.')

    def status(self):
        print(f'Pending orders: {len(self.orders)}')

cq = CafeteriaQueue()
cq.place_order('Muneer',  'Biryani')
cq.place_order('Ali',     'Dosa')
cq.place_order('Priya',   'Sandwich')
cq.serve_order()
cq.status()
```

```
Order placed: Muneer -> Biryani
Order placed: Ali -> Dosa
Order placed: Priya -> Sandwich
Serving: Muneer -> Biryani
Pending orders: 2
```

Result:
Order placed: Muneer → Biryani | Ali → Dosa | Priya → Sandwich
Serving: Muneer → Biryani | Pending orders: 2

Observation: Python's *deque* was the ideal structure for the cafeteria queue as it provides O(1) enqueue (*append*) and dequeue (*popleft*) operations. FIFO ordering ensures fairness —

the first order placed is the first to be served. The *status()* method gives real-time visibility into the queue length. This pattern directly maps to real-world food ordering and ticketing systems.